

Security Vulnerability Assessment Report

Critical Findings

1. CRITICAL: Password Hash Logged to Console

File:

password_reset.go

Issue: The hashed password is logged using `logger.Info()`, which will write sensitive data to logs that may be accessible to operators or log aggregation systems.

// Line 220-221

```
logger.Info("Hashed password", zap.String("hashed_password", hashedPassword))
```

Impact: Even though hashed, this exposes the password hash which could be used for offline cracking attacks if logs are compromised.

Fix: Remove this debug log statement entirely.

- `logger.Info("Hashed password", zap.String("hashed_password", hashedPassword))`

2. HIGH: Weak Password Minimum in ChangePassword

File:

user_handler.go

Issue: The

`ChangePasswordRequest` struct only enforces a `min=6` character password, while registration and password reset correctly enforce `min=12`.

```
type ChangePasswordRequest struct {
```

```
    OldPassword string `json:"old_password" validate:"required"`
```

```
    NewPassword string `json:"new_password" validate:"required,min=6" // WEAK!
```

```
}
```

Comparison:

- `SetPasswordRequest` (line 38): `min=12` ✓
- `ForgotPasswordResetRequest` (line 55): `min=12` ✓

Impact: Users can change to a weaker password (6 chars) after registration, bypassing the strong password policy.

Fix: Change to `min=12` and add password strength validation in the handler:

```
type ChangePasswordRequest struct {
```

```
    OldPassword string `json:"old_password" validate:"required"`
```

```
- NewPassword string `json:"new_password" validate:"required,min=6"
```

```
+ NewPassword string `json:"new_password" validate:"required,min=12"
```

```
}
```

Additionally, add `utils.ValidatePasswordStrength()` call in the handler or service layer to ensure consistency with password reset.

3. HIGH: User Enumeration via UID Lookup

File:

internal_transfer in wallet_service.go

Issue: The

InternalTransfer function returns different error messages for invalid UID vs email lookups:

- "receiver not found" when user doesn't exist

Combined with public UID disclosure in transfer descriptions (line 311), this allows enumeration of valid UIDs.

// Line 311

Description: fmt.Sprintf("Internal transfer from user %d", senderID),

Impact: Attackers can brute-force UIDs to discover valid user accounts.

Fix: Return generic error messages and avoid disclosing internal IDs:

- Description: fmt.Sprintf("Internal transfer from user %d", senderID),

+ Description: "Internal transfer received",

4. MEDIUM: Missing IDOR Protection on Admin User Routes

File:

router.go

Issue: Admin routes like GET /users/:id and DELETE /users/:id only check for admin role but don't prevent actions on super-admin accounts or self-deletion.

Impact: An admin could potentially delete another admin or view/modify privileged user data.

Fix: Add checks in handlers to prevent:

1. Self-deletion
 2. Actions on users with equal or higher privileges
-

5. MEDIUM: P2P Ad Creation Lacks Cross-User Order Validation

File:

p2p_service.go

Issue: The P2P system currently only has ad creation. When order matching is implemented, ensure:

1. Users cannot trade with themselves
2. Rollover ads properly check real-time balance during order execution (race condition risk)
3. Escrow wallet isolation is enforced

Impact: Potential for self-trading manipulation and race conditions on balance checks.

Fix: Document these requirements and implement checks when order matching is added.

6. MEDIUM: Missing Account Lockout After Failed Login Attempts

Current State: Rate limiting exists globally via IP, but there's no per-account lockout after multiple failed login attempts.

Impact: Attackers can distribute credential stuffing attacks across multiple IPs.

Fix: Implement per-email rate limiting similar to OTP verification (track failed attempts in Redis, lock account temporarily after N failures).

Positive Security Controls Found

Control	Status	Location
JWT Algorithm Verification	✓ HMAC-only	jwt.go:39-42
Webhook HMAC Signature	✓ Verified	blockradar_handler.go:47
Webhook Idempotency	✓ TX hash dedup	blockradar_webhook_service.go:97-106
SQL Injection Prevention	✓ GORM parameterized	All repositories
CORS Wildcard Filtering	✓ Production-only	cors.go:23-32
Body Size Limit	✓ 1 MiB	router.go:15, 63-66
Swagger Disabled in Prod	✓	router.go:82
SSRF Protection	✓ URL validation	config.go:305-350
JWT Secret Strength	✓ 32+ chars	config.go:287-302
OTP Rate Limiting	✓ Max 3 resends	registration_otp.go:20-21

OTP Brute-Force Protection	✓ Max 5 attempts	registration_otp.go:22
Constant-Time Comparison	✓ subtle.ConstantTime Compare	Multiple files
2FA Required for Transfers	✓ TOTP validation	wallet_service.go:192-205
Atomic Balance Updates	✓ Row-level locks	wallet_service.go:231-240
Password Strength Validation	✓ Complex rules	password_reset.go:192-193

Proposed Changes

[MODIFY]

password_reset.go

- Remove password hash logging (line 220-221)

[MODIFY]

user_handler.go

- Update ChangePasswordRequest.NewPassword validation from min=6 to min=12
- Add utils.ValidatePasswordStrength() validation in
- ChangePassword handler

[MODIFY]

wallet_service.go

- Replace sender ID exposure in transfer description with generic text

[NEW] Login attempt rate limiting (future enhancement)

- Add per-email failed login tracking in Redis

- Lock account temporarily after 5 failed attempts
-